Topic: *Orchest Computing (Orchestrator suy luận phân tán cho cụm thiết bị biên dị chủng)*
Programming Language: Python
Framework & Techstack: PyTorch, Transformers, CUDA, gRPC/ ZeroMQ, NVML (NVIDIA Management Library), WMI (Windows Management Instrumentation), System APIs (macOS), Mutual TLS (mTLS), Digital Certificates

---

Orchest Computing

Abstract

This project researches and develops **Orchest Computing**, a runtime orchestrator for distributed Large Language Model (LLM) inference across heterogeneous edge clusters. The system enables multiple consumer-grade devices within a Local Area Network (LAN) to collaboratively execute pipeline-parallel inference through automatic device discovery, real-time resource telemetry, contiguous layer partitioning, and multi-node execution coordination.

The research focuses on the distributed inference runtime problem, encompassing model partitioning, heterogeneous scheduling, and runtime orchestration. The core objective is to formulate a model partitioning mechanism that jointly considers computational capacity, memory limits, network communication characteristics, and other performance factors to optimize inference performance across heterogeneous edge devices. Within an 18-week timeframe, a prototype is implemented on Windows and macOS using PyTorch, CUDA, MPS, and evaluated comprehensively across latency, throughput, and resource utilization metrics.

1. Research Background and Motivation

The rapid advancement of foundational models (such as Llama, Qwen, and Gemma) has driven substantial demand for local inference on personal devices. However, the parameter scale and memory footprint of modern LLMs frequently exceed the memory capacity of a single machine. Consequently, harnessing multiple co-located devices over a local network has emerged as a crucial research direction.

In recent years, systems such as *exo*, *Petals*, *PipeEdge*, and *PyTorch Pipeline Parallelism* have demonstrated the feasibility of distributed inference via pipeline parallelism and model partitioning. These frameworks have introduced mechanisms such as automatic peer discovery, resource tracking, and topology-aware model partitioning. Nonetheless,

notable research gaps remain—particularly in designing dedicated cost models tailored to the distinct **Prefill** and **Decode** phases of LLMs, optimizing partitioning under joint constraints (model weights, activation memory, and KV-cache footprints), and evaluating the real-world overhead of heterogeneous hardware environments across local networks.

2. Proposed Solution and System Architecture

Orchest Computing adopts an **Orchestrator–Worker** architecture, where a designated leader node orchestrates the end-to-end inference lifecycle while worker nodes execute assigned sub-segments of the model. Upon cluster bootstrap, each node continuously profiles and reports its hardware status—including CPU, GPU, available memory, thermal state (Thermal-aware Scheduling), and network link quality. Leveraging this telemetry, the Orchestrator constructs a global cluster cost model and determines an optimal model partitioning plan.

Within the scope of this project, the Transformer model is partitioned using **contiguous layer partitioning**, where each participating node hosts a contiguous slice of consecutive layers. This strategy aligns naturally with pipeline parallelism while significantly reducing inter-node activation communication compared to fine-grained operator-level partitioning. To eliminate runtime migration overhead, the system performs dynamic repartitioning strictly at session boundaries (after an active request has completed). Advanced techniques—such as live migration, operator-level placement, and intra-layer tensor parallelism—are designated as future extensions.

3. Optimization Model and Methodology

The model partitioning challenge is formulated as an optimization problem aimed at minimizing **Time-to-First-Token (TTFT)** during the **Prefill** phase. The Prefill phase processes the full input prompt sequence once per request; minimizing its execution time directly reduces user-perceived initial latency. Within this scope, a **Dynamic Programming (DP)** algorithm is developed to search for the partition that minimizes Prefill TTFT rather than jointly optimizing both TTFT and Decode throughput.

A partition is determined at the beginning of a session and held constant throughout the entire request lifecycle (covering both Prefill and Decode phases). Repartitioning occurs only between sessions upon request completion. Thus, optimizing for the Prefill stage introduces zero runtime migration overhead for model weights or KV-cache states within a request. While Decode execution runs on the chosen partition, Decode throughput is not treated as a primary objective in the formulation.

The optimization is subject to the VRAM budget of each node, activation memory requirements, dynamic KV-cache footprint (governed by context length and batch size), and inter-node activation transfer latency. For the MVP, the node topology in the pipeline is assumed to be static, with each node hosting a single contiguous block of layers.

For a model with L layers distributed across a cluster of K nodes, a partition is represented by a set of $K - 1$ cut points:

$$P = (p_0, p_1, \ldots, p_K)$$

subject to:

$$0 = p_0 \leq p_1 \leq \ldots \leq p_K = L$$

Node k is allocated layers from $p_{k-1} + 1$ to p_k. If two adjacent cut points coincide ($p_{k-1} = p_k$), node k is assigned zero layers (bypassed). This formulation allows the DP algorithm to comprehensively explore all valid partitioning configurations.

For each stage, the Prefill stage cost combines the computation time of the assigned layer block and the communication time required to receive activations from the preceding stage:

$$C_{pre}(k, i, j) = T^{pre}_{compute}(k, i, j) + T^{pre}_{comm}(k - 1, k, i)$$

Where $T^{pre}_{compute}(k, i, j)$ represents the Prefill computation time on node k, and $T^{pre}_{comm}(k - 1, k, i)$ represents the activation transfer time from node $k - 1$ to node k. Communication cost is determined based on activation size at the boundary and actual bandwidth measured between the two nodes. This formulation of the Cost Model allows decoupling the influence of computational capacity and network characteristics on TTFT.

For node k, the total memory requirement for layers i through j is modeled as:

$$M(k, i, j) = M_{weights}(k, i, j) + M_{KV-cache}(k, i, j) + M_{activation}(k, i, j) + M_{runtime}(k)$$

A candidate stage assignment is feasible if and only if:

$$M(k, i, j) \leq VRAM(k)$$

Here, KV-cache is determined based on context length and batch size of the workload. If the memory requirement of a stage exceeds the available memory capacity on the corresponding node, that stage is marked as infeasible and is not considered during the partition search process.

Let $DP[k][j]$ be the minimum TTFT when partitioning the first j layers across the first k nodes. When considering a cut point $i < j$, node k will handle the layer segment from $i + 1$ to j. If this layer segment satisfies the memory constraint, the state is updated according to the recurrence:

$$DP[k][j] = \min_{i<j} \{ DP[k-1][i] + C_{pre}(k, i, j) \}$$

Where only values of i satisfying:

$$M(k, i, j) \leq VRAM(k)$$

are used to update the state. If no feasible cut point exists, the state $DP[k][j]$ is marked as infeasible.

However, because the system uses pipeline parallelism, the actual TTFT of Prefill depends not only on the total computation time of the stages, but also on execution order and data transfer time between stages. Therefore, the Cost Model will model compute time and communication time separately at each stage, then use profiling results to estimate the completion time of the entire Prefill pipeline. The ultimate goal of Dynamic Programming is to select the set of cut points yielding the smallest TTFT among feasible partitions.

To evaluate partition quality, the Dynamic Programming method will be compared against heuristics such as Equal-layer Partition, Memory-proportional Partition, and Compute-aware Partition. On small clusters, the study will perform an exhaustive search across all feasible partitions to construct a measured oracle, thereby determining the partition with the lowest actual TTFT. The result of Dynamic Programming is compared

with the measured oracle via TTFT and partition regret to assess its proximity to the actual optimal solution.

With L layers and K nodes, Dynamic Programming has a time complexity of $O(KL^2)$ when examining all possible cut points, and a space complexity of $O(KL)$. Stage costs are precomputed via the Cost Model to reduce computational overhead during the partition search process.

Problems regarding optimizing Decode throughput, multi-objective optimization between TTFT and Decode throughput, Pareto Frontier generation, topology optimization, node order selection, live repartitioning, tensor parallelism, and operator-level partitioning will be reserved for subsequent studies.

4. Implementation Technologies

The prototype is implemented in Python on top of PyTorch and HuggingFace Transformers, utilizing a **Backend Abstraction Layer** for heterogeneous hardware interoperability:

- **Windows Platforms:** Resource telemetry and compute execution are handled via CUDA, NVML, and WMI.
- **macOS Platforms:** System state is profiled via native system APIs, and compute workloads execute via PyTorch's Metal Performance Shaders (MPS) and MLX backends.

A modular **Transport Abstraction Layer** decouples tensor serialization and network protocols from the core execution engine. During initial development, **gRPC** and **ZeroMQ** are benchmarked under identical network conditions regarding latency, throughput, serialization overhead, tensor payload efficiency, and connection recovery to select the primary communication backbone.

5. Security Architecture

The prototype secures inter-node communication through:

- **Mutual TLS (mTLS):** Cryptographic mutual authentication and end-to-end transport layer encryption for all tensor exchanges.
- **Device Identity & Digital Certificates:** Ensuring only authenticated devices join the inference cluster.
- **Allowlist Enforcement:** Strict node access control.

- **Tensor Integrity Checks:** Boundary payload hashing/checksums to verify data integrity in transit.

6. Timeline

The research is carried out over an 18-week duration and divided into four consecutive phases to ensure systematic development, spanning theoretical survey, distributed inference platform construction, empirical evaluation, and prototype finalization.

In the first phase (Weeks 1–4), the research focuses on surveying related work on distributed inference, pipeline parallelism, and model partitioning. Representative systems such as exo, Petals, PipeEdge, and PyTorch Pipeline Parallelism are analyzed and synthesized into a Literature Matrix to identify research gaps and project contributions. Concurrently, the overall system architecture is designed, the Transformer model is analyzed to construct the contiguous layer partitioning strategy, the development environment is set up, and communication frameworks like gRPC and ZeroMQ are evaluated to select the appropriate solution for the system.

In the second phase (Weeks 5–9), core components of the distributed platform are developed. The system sequentially implements Auto Discovery, Node Registry, Heartbeat, Node Profiling, and Resource Monitor mechanisms to collect information on CPU, GPU, RAM, VRAM, and network conditions on Windows and macOS devices. In parallel, the Cost Model for both Prefill and Decode phases is constructed along with the KV-cache model, establishing the foundation for designing and implementing the Dynamic Programming algorithm to solve the contiguous layer partitioning problem.

In the third phase (Weeks 10–14), all components are integrated into a complete distributed inference runtime. The system deploys the Pipeline Parallel Runtime, Transport Abstraction layer, Asynchronous Execution mechanism, Scheduling module, and Session-level Repartition mechanism. Following integration, the system is tested on heterogeneous two-node and three-node clusters to evaluate coordination among components, tensor transmission efficiency, and adaptability when cluster resources or states change.

In the final phase (Weeks 15–18), the prototype undergoes comprehensive evaluation through the designed benchmark suite. Experiments are conducted across different partitioning strategies for comparison against the proposed method, accompanied by

statistical analysis, system stability evaluation, and auxiliary experiments such as ablation studies and analysis of the effects of network conditions, computational load, and device temperature.

Finally, the prototype is finalized, technical documentation and the project report are compiled, and preparations are made for presentation and research evaluation.

7. Non-Functional Requirements

The system is designed to ensure correctness of inference results, execution performance, and incident handling capabilities in heterogeneous edge environments

a/ Numerical Correctness

Model partitioning and tensor transmission across nodes must not significantly alter inference results. The system must meet the following requirements:

- Distribution-level Evaluation:
    - Use Teacher Forcing on fixed input sets for direct comparison.
    - Measurement metrics include: Max and Mean Absolute Logit Error, Relative Error.
    - Cosine Similarity and KL Divergence distance between probability distributions.
    - CI values within acceptable tolerance thresholds.
- Generation Agreement:
    - Measure Top-1 / Top-k agreement and Greedy-token agreement on complete generation sequences.
- Transport Boundary Integrity:
    - Integrate Checksums on tensors before sending and after receiving to eliminate silent corruption risks.
    - Enforce NaN and inf checks immediately at payload decoding to prevent error propagation within the pipeline.
-

b/ Performance

System performance is evaluated through metrics reflecting the end-to-end inference process:

- Time-to-First-Token (TTFT).
- Inter-token Latency.
- Throughput (tokens/s).
- p50 Latency.
- p95 Latency.

c/ Fault Handling

Within the MVP scope, the system applies a Safe Failure mechanism instead of Live Recovery to avoid the complexity of replication and checkpointing. When a node experiences a failure:

- The current request is safely aborted.
- The runtime re-profiles the entire cluster.
- The partitioning algorithm is recomputed.
- Model weights are loaded onto remaining active nodes.
- The system resumes serving new requests without needing to restart the whole service.

8. Benchmark Design

To comprehensively evaluate the system, experiments are designed to address two core Research Questions (RQs) regarding the limits and costs of the distributed runtime:

- RQ1 (Scaling Beyond Limits): Can the system coordinate and successfully operate models larger than the total VRAM capacity of any individual node in the cluster?
- RQ2 (Distributed Overhead): For small models that fit entirely on one node, how much faster or slower is inference performance over the distributed runtime (impacted by LAN communication and pipeline synchronization) compared to the single-machine baseline?
- RQ3 (Worker Capacity): Can the system execute models/workloads that exceed the memory capacity of individual nodes?
- RQ4 (Fidelity): Does distributed inference preserve numerical and generation correctness?
- RQ5 (Partition Quality): How much does DP improve TTFT compared to heuristics, and how close does it get to the measured oracle?
- RQ6 (Distributed Cost): When does distributed inference reach a break-even point compared to single-node/offload?

- RQ7 (Cost Model Fidelity): How accurately does the cost model predict compute, communication, and memory on unseen workloads? Usable metrics: MAPE/RMSE of predicted stage time, VRAM/unified-memory peak prediction error, Spearman correlation between predicted and actual measured partition ranks, and finally partition regret.

8.1. Baselines

- Single-node inference: baseline for latency/throughput when the model fits on a single node.
- Equal-layer Partition.
- Memory-proportional Partition.
- Compute-aware Partition.
- Measured Oracle: exhaustive search on small clusters to determine the best partition according to the objective.
- Single-node/offload baseline when the model exceeds VRAM but can run via offloading mechanisms

Benchmark Matrix

Algorithm and system architecture efficacy is cross-evaluated across a comprehensive parameter matrix to reflect all practical operational conditions:

- Infrastructure & Network: Cluster size, network bandwidth/latency conditions, and device background load.
- Workload Specification: Model families and parameter sizes, input sequence length, output sequence length.
- Execution Scope: Required concurrency level, batch size, and microbatch size.
- System Strategies: Comparison of partitioning algorithms and evaluation of security overhead (latency when enabling/disabling mTLS and Tensor Checksum).
- Partition Analysis: Number of layers per stage, stage bottlenecks, pipeline bubbles, and communication volume.

Comparative Methods and Measurement Metric Groups

The proposed Dynamic Programming (DP) model partitioning method will be compared directly against: Single-node Inference (used as baseline for RQ2), Equal-layer Partition, Memory-proportional Partition, and Compute-aware Partition. The evaluation process is performed across three metric groups:

- Inference Performance: Time-to-First-Token (TTFT), Inter-token Latency, Throughput (tokens/s), p50 Latency, and p95 Latency.
- Resource Utilization: CPU Utilization, GPU Utilization, RAM/VRAM consumption, and device temperature progression during operation.
- System Overhead: Total volume of data transferred between nodes, scheduling algorithm execution time, resource wastage from waiting (pipeline bubble), and communication overhead incurred at network boundaries.

Experiment procedure

- All experiments are conducted in strictly controlled environments. Scenarios employ the same model, the same prompt set, fixed sequence lengths, and consistent network configurations.
- All measurement processes undergo a warm-up phase to eliminate cold-start effects and share a common random seed to ensure reproducibility.
- Each measurement is repeated at least 10 times, with final results reported as Mean ± 98% CI (Confidence Interval) to ensure statistical significance and experimental reliability.

9. Future Work

Following prototype completion, the research can be extended along several directions to enhance the adaptability and performance of the distributed inference runtime. First, the system can incorporate an Online Repartition mechanism combined with Live Migration to alter partition points during inference without terminating the active session.

Concurrently, expanding from contiguous layer partitioning to Tensor Parallelism or Operator-level Placement will help exploit diverse cluster configurations more effectively. In addition, the runtime can include a Reliability-aware Scheduling mechanism, and automatic failure recovery based on checkpoints or replication will serve as important research directions to enhance system fault tolerance.

Finally, a promising research avenue is integrating machine learning models into the orchestrator to forecast fluctuations in computational load, device temperature, or network bandwidth. These predictions can be leveraged to proactively adjust partitioning and scheduling plans, thereby mitigating bottlenecks and elevating system performance in practical deployment environments